



人工智能原理与算法

第7章-逻辑Agent

中国科学院自动化研究所
国科大人工智能学院

雷震

- **基于知识的Agent**
- **Wumpus世界**
- **逻辑**
- **命题逻辑：一种简单的逻辑**
- **命题逻辑定理证明**
- **高效的命题逻辑模型检验**
- **基于命题逻辑的Agent**

7.1 基于知识的Agent

■ 基于知识的Agent

- 人类一直强调人的智能是如何获得——不是靠反射机制而是**对知识的内部表示进行操作的推理**过程；人工智能界，这种智能方法体现在**基于知识的Agent**上
- 基于知识的Agent能够以显示描述目标的形式接受新任务；通过被告知或者主动学习环境的新知识从而快速获得能力；通过更新相关知识适应环境的变化

7.1 基于知识的Agent

■ 基于知识的Agent

- 基于知识的Agent的核心部件是知识库，或称KB
- 知识库是一个语句集合，这些语句表示关于世界的某些断言
- 公理：直接给定而不是推导得到
- TELL（告诉）和ASK（询问）：将新语句添加到知识库以及查询目前所知内容的方法

7.1 基于知识的Agent

■ 基于知识的Agent程序轮廓

- 输入：感知信息
- 输出：一个行动
- Agent维护一个知识库KB，初始化时包括背景知识

```
function KB-AGENT(percept) returns 一个action  
  persistent: KB, 一个知识库  
             t, 一个计数器, 初始为0, 表示时间  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action  $\leftarrow$  ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t  $\leftarrow$  t + 1  
  return action
```

图 7-1 通用的基于知识的智能体。给定一个感知，智能体将这一感知添加进知识库，向知识库询问最优动作，并告知知识库它已经采取了这一动作

7.1 基于知识的Agent

■ 基于知识的Agent程序轮廓

- Agent告诉 (TELL) 知识库它感知到的内容
- 询问 (ASK) 知识库应该执行什么行动 (在回复该查询的过程中, 可能要对关于世界的当前状态、可能行动序列的执行结果等进行大量推理)
- Agent程序用TELL告诉知识库它所选择的行动, 并执行该行动

```
function KB-AGENT(percept) returns 一个action  
  persistent: KB, 一个知识库  
           t, 一个计数器, 初始为0, 表示时间  
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))  
  action  $\leftarrow$  ASK(KB, MAKE-ACTION-QUERY(t))  
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))  
  t  $\leftarrow$  t + 1  
  return action
```

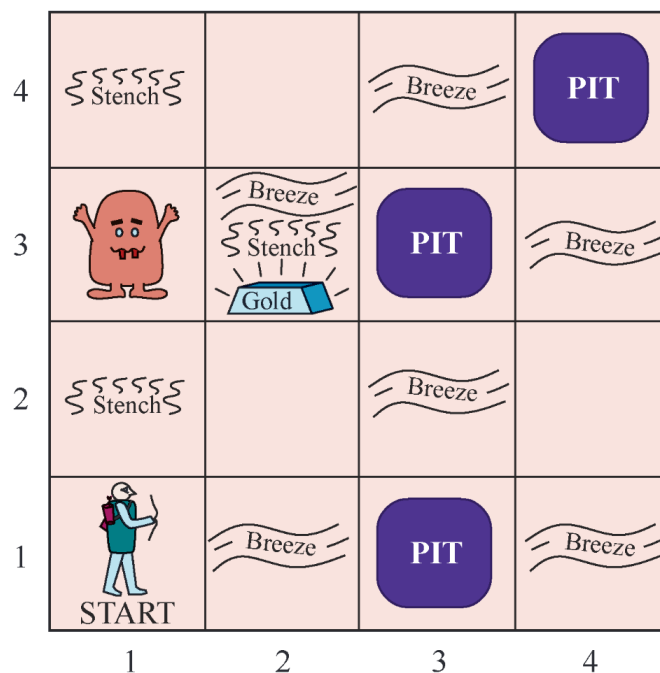
图 7-1 通用的基于知识的智能体。给定一个感知, 智能体将这一感知添加进知识库, 向知识库询问最优动作, 并告知知识库它已经采取了这一动作

- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.2 Wumpus世界

■ Wumpus世界

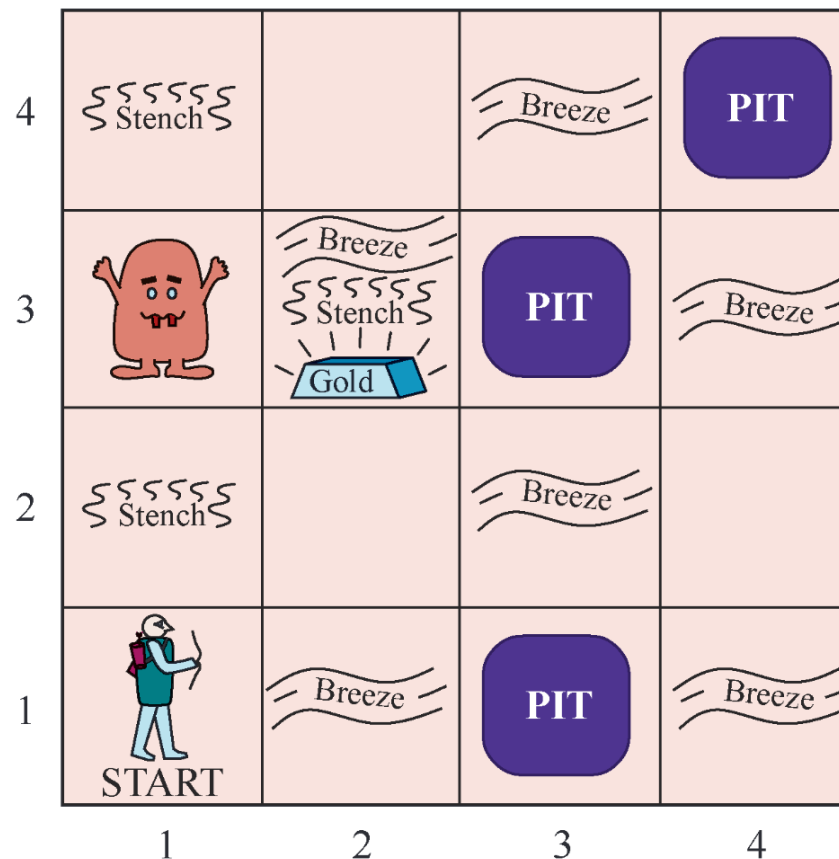
- Wumpus世界由多个房间组成并相连接起来的山洞。在洞穴某处隐藏着一只Wumpus（怪兽），它会吃掉进入它房间的任何人。Agent可以射杀Wumpus，但是Agent只有一枚箭。某些房间是无底洞，任何人漫游到这些房间都会被无底洞吞噬。生活在该环境下的唯一希望是存在发现一堆金子的可能性



7.2 Wumpus世界

■ 任务环境 (PEAS描述)

- Performance (性能)
- Environment (环境)
- Actuators (执行器)
- Sensors (传感器)



7.2 Wumpus世界

■ 任务环境（PEAS描述）

- **性能度量**：带着金子爬出洞口+1000，掉入无底洞或者被Wumpus吃掉得-1000，每采用一个行动得-1，而用掉箭-10。
游戏在Agent死亡或Agent出洞时结束
- **环境**：4×4的房间网格。Agent每次都从标号为[1,1]的方格出发，面向右方。金子和Wumpus的位置按均匀分布随机选择除了起始方格以外的方格。另外，除了起始方格以外的任一方格都可能是无底洞，概率为0.2

7.2 Wumpus世界

■ 任务环境（PEAS描述）

□ 执行器：

- Agent 可以向前移动、左转90°或者右转90°。如果它碰上无底洞或者活着的Wumpus,它将悲惨地死去（进入死Wumpus 的方格是安全的，尽管很臭）。
- 如果Agent前方是一堵墙，那么不能向前移动。
- 行动Grab可以用于捡起Agent所处方格内的物体。
- 行动Shoot可以用于向Agent 所正对的方向射箭。箭向前运动直到击中Wumpus(此时Wumpus将被杀死) 或者击中墙。Agent 只有一支箭，因此只有第一个Shoot行动有效。
- 最后，行动Climb用于爬出洞口，只能从方格[1,1]中爬出。

7.2 Wumpus世界

■ 任务环境（PEAS描述）

- **传感器：** Agent 具有五个传感器，每个都可以提供一些单一信息。
 - 在Wumpus 所在之处以及与之直接相邻（非对角的）的方格内， Agent 可以感知到臭气
 - 在与无底洞直接相邻的方格内， Agent 能感知到微风
 - 在金子所处的方格内， Agent 感知到闪闪金光
 - 当Agent 撞到墙时，它感知到撞击
 - 当Wumpus被杀死时，它发出的悲惨嚎叫在洞穴内的任何地方都可以感知到
- 这5 种感知信息以符号列表形式提供给Agent; 例如，如果有臭气和微风，但是没有金光、撞击或者嚎叫，那么Agent 接收到的感知信息为
[Stench, Breeze, None, None, None]

7.2 Wumpus世界

■ Wumpus环境特征

- 此环境是离散的、静态的、单个Agent的（幸运的是怪兽不移动）
- 它是序列的，因为可能需要采取很多个行动后才会有后果产生
- 它是部分可观察的，因为状态的如下方面不能直接感知
 - ▶ Agent的位置、Wumpus的健康状态、箭是否还有效
- **此环境中的Agent，主要困难在于开始时它对环境配置一无所知；为了克服这种无知，需要逻辑推理**

7.2 Wumpus世界

- 假设移动到标有OK的方格[2,1]，感知到微风，基于前述环境规则，判断[2,2]、[3,1]可能有无底洞，返回[1,1]，前进到[1,2]

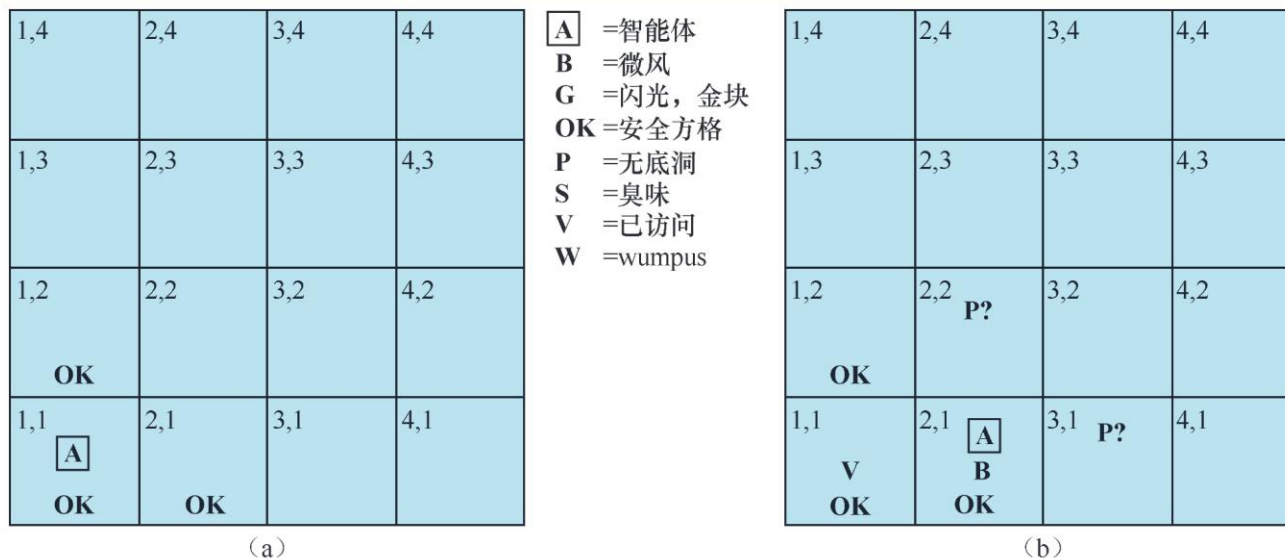


图 7-3 智能体在 wumpus 世界迈出的第一步。(a) 在感知到 [None, None, None, None, None] 后的初始状态。(b) 在移动到 [2, 1] 后感知到 [None, Breeze, None, None, None]

7.2 Wumpus世界

- 在[1,2]中感知到臭气，根据前述环境规则及上一步推理结果，判断Wumpus在[1,3]，[2,2]是安全的，[3,1]是无底洞

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A =智能体
B =微风
G =闪光，金块
OK =安全方格
P =无底洞
S =臭味
V =已访问
W =wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

(a)

(b)

图 7-4 智能体运作时的两个后续状态。(a) 回到 [1, 1] 再移动到 [1, 2] 后，感知到 [Stench, None, None, None, None]。(b) 来到 [2, 2] 再移动到 [2, 3]，感知到 [Stench, Breeze, Glitter, None, None]

7.2 Wumpus世界

- **逻辑推理**的本质：在每种情况下，Agent根据可用信息得出结论，如果可用信息正确，那么该结论能确保是正确的
- 接下来描述如何建造可以表示必要信息并推理得出结论的**逻辑Agent**

- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.3 逻辑

■ 逻辑

- 知识库由**语句**构成。根据表示语言的语法来表达这些语句，**语法**为合法语句给出规范。“ $x + y = 4$ ”是合法语句，而“ $x4y +=$ ”则不是
- **逻辑**定义语言的语义，**语义**定义了每个语句在每个可能世界的真值
算数的语义规范了语句“ $x + y = 4$ ”，在 x 等于2， y 也等于2的世界中为真；在 x 等于1， $y=1$ 的世界中为假
- **标准逻辑中**，每个语句在每个可能世界中非真即假——不存在“中间状态”

7.3 逻辑

■ 模型

- **模型**是数学抽象，每个模型只是简单地关注于每个相关语句的真或假
- 例如可能世界中 x 个男性和 y 个女性正坐在桌子旁玩桥牌，当总共有四个人时候，语句 $x + y = 4$ 为真。**可能模型就是对变量 x 和 y 的所有可能赋值**
- 语句 α 在模型 m 中为真，称 m 满足 α ，有时也称 m 是 α 的一个模型，使用 $M(\alpha)$ 表示所有模型

7.3 逻辑

■ 蕴涵关系

- 语句间的逻辑蕴涵 (entailment) 关系——某语句逻辑上跟随另一个语句，数学符号表示为

$$\alpha | = \beta$$

- 意为语句 α 蕴涵语句 β 。蕴涵的形式化定义：当且仅当在使 α 为真的每个模型中， β 也为真。可以记为

$$\alpha | = \beta \text{ 当且仅当 } M(\alpha) \subseteq M(\beta)$$

- 若 $\alpha | = \beta$ ，则 α 是比 β 更强的断言：排除了更多的可能世界
- 蕴涵关系与算术类似：语句 $x = 0$ 蕴含了语句 $xy = 0$ 。显然，在任何 $x = 0$ 的模型中， xy 的值都是0，不管 y 的值是多少

7.3 逻辑

■ 举例：Wumpus世界

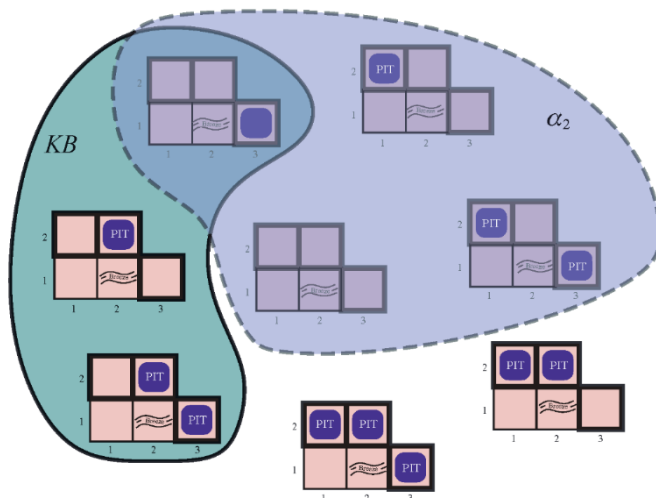
- Agent在[1,1]中未检测到任何异常，而[2,1]中有微风
- [1,2], [2,2]和[3,1]是否有无底洞？ 共存在8种情况（模型）
- 在与Agent所知道的内容相矛盾的模型中，KB为假

- ▶ 例如，在任意[1,2]包含无底洞的模型中，KB为假，因为[1,1]没检测到微风
- ▶ 实际上只有三个模型使得KB为真（实线标出）

$\boxed{A}$ =智能体
 B =微风
 G =闪光，金块
 OK =安全方格
 P =无底洞
 S =臭味
 V =已访问
 W =wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 $\boxed{A}$ B OK	3,1 P?	4,1

(b)



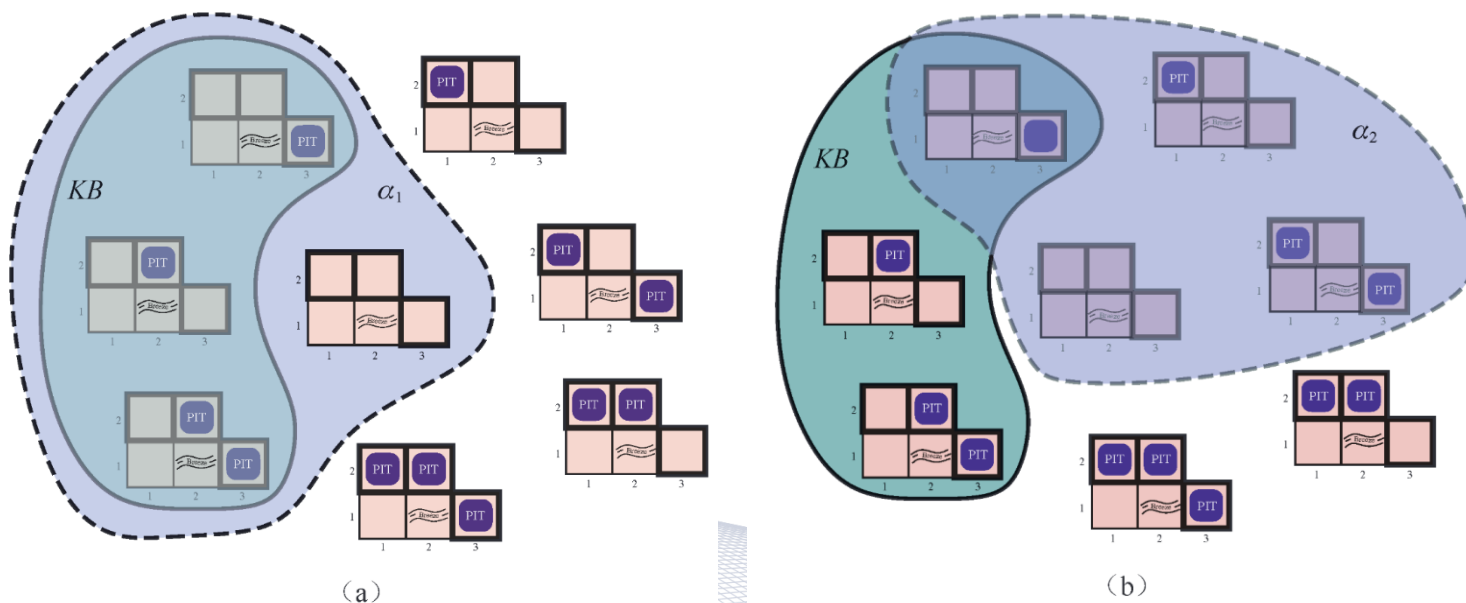
(b)

- 两个可能的结论：
- α_1 = “[1,2]中没有无底洞”
- α_2 = “[2,2]中没有无底洞”

7.3 逻辑

■ 举例：Wumpus世界

- 下图用虚线分别标出了 α_1 和 α_2 的模型。通过检验可以得到以下结果：
 - ▶ 在KB为真的每个模型中， α_1 也为真。因而， $KB \models \alpha_1$ ：[1,2]中没有无底洞。
 - ▶ 在KB为真的某些模型中， α_2 为假，因而， $KB \not\models \alpha_2$ ：Agent无法得出[2,2]没有无底洞的结论



7.3 逻辑

■ 蕴涵和推理算法

- 蕴涵就像是干草堆里的一根针；推理就像寻找它的过程，形式表示：如果推理算法*i*根据KB导出 α ，记为 $KB| =_i \alpha$ 。读为“ α 通过*i*从KB导出”或者“*i*从KB导出 α ”
- 一个仅推导蕴含语句的推断算法被称为是**可靠的**或**保真的**。
- 如果一个推断算法能够推导出所有蕴含的语句，则它是**完备的**。
- 最后要考虑的问题是**落地**，也就是逻辑推理过程与智能体所存在的真实环境的联系。

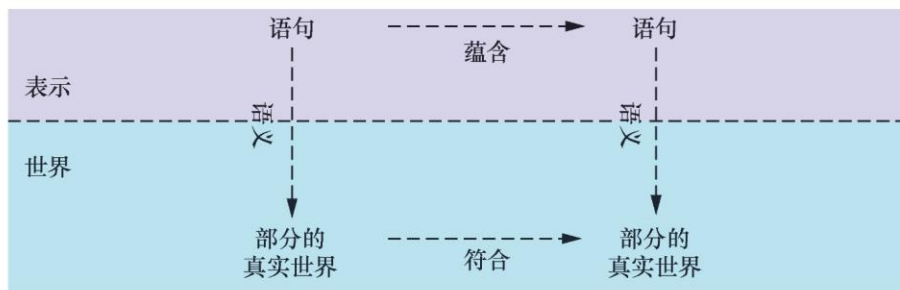


图 7-6 语句是智能体的物理结构，而推理是从旧结构构建新结构的过程。逻辑推理应当确保新结构所表示的部分世界确实能够从旧结构所表示的部分世界推得

- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.4 命题逻辑-语法

■ 命题逻辑的语法

- **原子语句**由单个**命题词**组成。每个命题词代表一个或为真或为假的命题，采用大写字母表示
 - ▶ 可能包括其他符号或下标： P 、 Q 、 R 、 $W_{1,3}$ 和North等
 - ▶ 有两个命题词有固定含义：True是永真命题，False为永假命题
- **复合句**由简单语句用**括号**和**逻辑连接词**构成

7.4 命题逻辑-语法

■ 常用的五种逻辑连接词：

- $\neg$ (**非**) , 如 $\neg W_{1,3}$ 被称为 $W_{1,3}$ 的否定式。文字指原子语句 (正文字) 或原子语句的否定式 (负文字)
- $\wedge$ (**与**) , 如 $W_{1,3} \wedge P_{1,3}$, 被称为合取式, 各部分称为合取子句
- $\vee$ (**或**) , 如 $(W_{1,3} \wedge P_{1,3}) \vee W_{2,2}$, 是析取字句 $W_{1,3} \wedge P_{1,3}$ 和 $W_{2,2}$ 的析取式
- $\Rightarrow$ (**蕴含**) $W_{1,3} \wedge P_{1,3} \Rightarrow \neg W_{2,2}$ 的语句称为蕴涵式 (implication) (或条件式)

7.4 命题逻辑-语法

■ 常用的五种逻辑连接词：

□ $\Rightarrow$ (**蕴含**) 它的前提或称前项是 $W_{1,3} \wedge P_{1,3}$ ，结论称后向为 $\neg W_{2,2}$ 。

蕴含式同时也称为规则或 if-then 语句，有些书中记为 $\supset$ 或 $\rightarrow$

□ $\Leftrightarrow$ (**当且仅当**)：语句 $W_{1,3} \Leftrightarrow \neg W_{2,2}$ 是双向蕴含式，有些书记为 $\equiv$

□ 运算符优先级：“否定”运算符 $\neg$ 具有最高优先级

$\neg A \wedge B$ 等价于 $(\neg A) \wedge B$ 而不是 $\neg(A \wedge B)$

7.4 命题逻辑-语义

- 语义定义了用于判定特定模型中的语句真值的规则
- 命题逻辑固定了每个命题词的真值——true或false
 - 例如，知识库中的语句采用命题词 $P_{1,2}$ 、 $P_{2,2}$ 和 $P_{1,3}$ ，那么一个模型可能是

$$M1 = \{P_{1,2} = false, P_{2,2} = false, P_{1,3} = false\}$$

- 命题逻辑的**语义**需要规范在已知模型下如何计算任一语句的真值，通过递归来实现
 - 所有语句都是由原子语句和5种连接词构成
 - 需要规范如何计算原子语句的真值和如何计算由5种连接词形成的语句的真值

7.4 命题逻辑-语义

- 复合句有5条规则，其中P和Q为任意子句， m 为任一模型
 - 在模型 m 中 $\neg P$ 为真，当且仅当 P 在 m 中为假
 - 在模型 m 中 $P \wedge Q$ 为真，当且仅当 P 和 Q 在 m 中都为真
 - 在模型 m 中 $P \vee Q$ 为真，当且仅当 P 或 Q 在 m 中都为真
 - 在模型 m 中 $P \Rightarrow Q$ 为真，除非 P 在 m 中为真且 Q 在 m 中为假
 - 在模型 m 中 $P \Leftrightarrow Q$ 为真，当且仅当 P 和 Q 在 m 中都为真或者都为假

7.4 命题逻辑-语义

■ 连接词的运算规则可以用真值表总结

- $\Rightarrow$ 的真值表可能不太符合人们对 “P 蕴涵 Q” 或 “若 P 则 Q” 的直观理解。一种解释是，命题逻辑并不要求 P 和 Q 之间有任何因果关系或相关性。

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

图 7-8 5 个逻辑联结词的真值表。若要使用真值表计算在 P 为真、 Q 为假时 $P \vee Q$ 的值，首先在左边找到 P 为 true 而 Q 为 false 的行（第 3 行），然后找到该行位于 $P \vee Q$ 列处的值，得到结果 true

7.4 命题逻辑-一个简单的知识库

- 基于命题逻辑的语义定义，为Wumpus世界构建一个知识库，对于每个位置 $[x, y]$ 需要如下命题词
 - 如果 $[x, y]$ 中有无底洞，则 $P_{x,y}$ 为真
 - 如果 $[x, y]$ 中有怪兽，则 $W_{x,y}$ 为真，不管是死是活
 - 如果在 $[x, y]$ 中感知到微风，则 $B_{x,y}$ 为真
 - 如果在 $[x, y]$ 中感知到臭气，则 $S_{x,y}$ 为真

7.4 命题逻辑-一个简单的知识库

- $[1,1]$ 中没有无底洞

$$R_1 : \neg P_{1,1}$$

- 一个方格里有微风，当且仅当在某个相邻方格中有无底洞。对于每个方格都应该说明这一情况：目前，只考虑相关方格：

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1}) \quad R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$$

- 前面的这些语句在所有的Wumpus世界中都为真。现在将Agent所处的特定世界中最初访问的两个方格的微风感知信息包括进来

$$R_4 : \neg B_{1,1} \quad R_5 : B_{2,1}$$

- 知识库由R1到R5的语句组成： $R_1 \wedge R_2 \wedge R_3 \wedge R_4 \wedge R_5$

7.4 命题逻辑-一个简单的推理过程

■ 一个简单的推理过程

- 目标是判断对于某些语句 α , $KB \models \alpha$ 是否成立

例如, $\neg P_{1,2}$ 是否可从现有KB得出

- 推理算法模型检验是对蕴涵定义的直接实现：枚举所有模型，验证 α 在KB为真的每个模型中为真
- Wumpus世界中相关命题词有7个，其中的三个模型，KB为真

7.4 命题逻辑-一个简单的推理过程

■ 一个简单的推理过程

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	false	false	false	false	false	false	true	true	true	true	false	false
false	false	false	false	false	false	true	true	true	false	true	false	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	true	true	false	true	true	false
false	true	false	false	false	false	true	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	false	true	true	true	true	true	<u>true</u>
false	true	false	false	false	true	true	true	true	true	true	true	<u>true</u>
false	true	false	false	true	false	false	true	false	false	true	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false	true	true	false	true	false

图 7-9 根据文中所述的知识库构建的真值表。如果从 R_1 到 R_5 都为 true，则 KB 为 true。这种情况在全部 128 行中只出现了 3 次（在最右侧的列中用下划线标出）。在这 3 行中， $P_{1,2}$ 均为 false，因此 $[1, 2]$ 中没有无底洞。但是， $[2, 2]$ 中可能有（也可能没有）无底洞

7.4 命题逻辑-一个简单的推理过程

■ 一个简单的推理过程

- 判定命题逻辑的蕴涵的一个通用算法
- 该算法是可靠的，因为直接实现了蕴涵的定义
- 完备的，因为可以用于任意 KB 和 α ，而且总能够终止

7.4 命题逻辑-一个简单的推理过程

function TT-ENTAILS?(KB, α) **returns** true或false

inputs: KB , 知识库, 一个命题逻辑的语句

α , 查询, 一个命题逻辑的语句

$symbols \leftarrow KB$ 和 α 中的命题符号列表

return TT-CHECK-ALL($KB, \alpha, symbols, \{\}$)

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** true或false

if EMPTY?($symbols$) **then**

if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)

else return true //当 KB 为false时, 始终返回true

else

$P \leftarrow \text{FIRST}(symbols)$

$rest \leftarrow \text{REST}(symbols)$

return (TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = \text{true}\}$)

and

 TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = \text{false}\}$))

图 7-10 用于确定命题蕴含的真值表枚举算法 (TT 代表真值表)。当语句在一个模型中成立, PL-TRUE? 返回 true。变量 $model$ 代表部分模型——对于部分符号的赋值。此处的关键字 **and** 不是命题逻辑中的运算符, 而是伪代码编程语言中的中缀; 如果其两个参数中的任意一个为 true, 则返回 true

- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.5 命题逻辑定理证明

- 通过**模型检验**判断蕴涵：枚举所有模型，并验证语句在所有模型中为真
- 如何通过**定理证明**判断蕴涵——在知识库的语句上直接应用推理规则以构建目标语句的证明，而无需关注模型
- 如果模型数目庞大而证明很短，那么定理证明就比模型检验更有效

7.5 命题逻辑定理证明

- **逻辑等价**：如果两语句 α 和 β 在同样的模型集中为真，则它们是逻辑等价的，即为 $\alpha \equiv \beta$

例如，用真值表很容易证明 $P \wedge Q$ 和 $Q \wedge P$ 逻辑等价

- 任意两语句 α 和 β 是等价的当且仅当他们互相蕴涵时， $\alpha \equiv \beta$ 当且仅当 $\alpha \models \beta$ 且 $\beta \models \alpha$

7.5 命题逻辑定理证明

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$ $\wedge$ 的交换律

$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$ $\vee$ 的交换律

$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$ $\wedge$ 的结合律

$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$ $\vee$ 的结合律

$\neg(\neg\alpha) \equiv \alpha$ 双重否定律

$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$ 假言易位

$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ 蕴涵消去

$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$ 等价消去

$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ 德摩根律

$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ 德摩根律

$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$ $\wedge$ 对 $\vee$ 的分配律

$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$ $\vee$ 对 $\wedge$ 的分配律

图 7-11 标准的逻辑等价。符号 α 、 β 、 γ 代表任意命题逻辑语句

7.5 命题逻辑定理证明

- **有效性**：一个语句是有效的，如果在所有模型中它都为真

例如，语句 $P \vee \neg P$ 为有效的

- 有效语句也被称为重言式，必定为真

- **演绎定理**

任意语句 α 和 β ， $\alpha \vdash \beta$ 当且仅当语句 $(\alpha \Rightarrow \beta)$ 是有效的

说明每个有效的蕴含语句都描述了一个合法的推理

- **可满足性**：如果一个语句在某些模型中为真，那么这个句子是可满足的

7.5 命题逻辑定理证明

■ 有效性和可满足性关联

- α 是有效的当且仅当 $\neg\alpha$ 不可满足；
- 对换过来看， α 是可满足的当且仅当 $\neg\alpha$ 不是有效的；
- $\alpha \models \beta$ 当且仅当语句 $\alpha \wedge \neg\beta$ 是不可满足的。

7.5 命题逻辑定理证明-推断与证明

- 肯定前件：应用推理规则得到一个证明——一系列结论直到目标语句，最著名的规则是**假言推理**规则，记为

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

- 只要给定任何形式 $\alpha \Rightarrow \beta$ 和 α 的语句，就可以推导出语句 β
- 例如，如果已知 $(WumpusAhead \wedge WumpusAlive) \Rightarrow Shoot$ 和 $(WumpusAhead \wedge WumpusAlive)$ ，那么就可以推导出Shoot

7.5 命题逻辑定理证明-推断与证明

- **消去合成词**，即可以从合取式推导出任何合取子句

$$\frac{\alpha \wedge \beta}{\beta}$$

- 例如，从 $(WumpusAhead \wedge WumpusAlive)$ 可以推导出 $WumpusAlive$ 。
- 通过考虑 α 和 β 的可能真值，很容易得出假言推理规则和消去合取词都是可靠的。于是这些规则可以用于任意实例，无须枚举所有模型就可以得到新的推理结论

7.5 命题逻辑定理证明-推断与证明

- 所有逻辑等价都可以作为推理规则

- 例：用于双向蕴含消去的等价给出两条推理规则

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta)(\beta \Rightarrow \alpha)} \text{ 和 } \frac{(\alpha \Rightarrow \beta)(\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

- 不是所有的推理规则都是两个方向都生效

- 例：无法按相反方向运用假言推理规则，无法从 β 得到 $\alpha \Rightarrow \beta$ 和 α

7.5 命题逻辑定理证明-推断与证明

■ 将上述推断规则和等价关系应用于wumpus世界

□ 从包含 R_1 到 R_5 的知识库，如何证明 $\neg P_{1,2}$ ，即证明 $[1,2]$ 中没有无底洞

(1) 对 R_2 使用等价消去，得到

$$R_6 : B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1}) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

(2) 对 R_6 使用合取消去，得到

$$R_7 : ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

(3) 假言易位逻辑等价关系得到

$$R_8 : (\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1}))$$

(4) 对 R_8 和感知 R_4 (即 $\neg B_{1,1}$) 使用肯定前件，得到

$$R_9 : \neg(P_{1,2} \vee P_{2,1})$$

(5) 使用德摩根律，得到结论

$$R_{10} : \neg P_{1,2} \wedge \neg P_{2,1}$$

也就是， $[1, 2]$ 和 $[2, 1]$ 都没有无底洞。

7.5 命题逻辑定理证明-推断与证明

- 上述证明过程也可以采用搜索算法来找出证明序列，证明问题可以定义为：
 - 初始状态：初始知识库
 - 行动：行动集合由应用于语句的所有推理规则组成，要匹配推理规则的上半部分
 - 结果：行动的结果是将推理规则的下半部分的语句实例加入数据库
 - 目标：包含要证明语句的状态

7.5 命题逻辑定理证明-推断与证明

- **单调性**：意味着逻辑蕴涵语句集合随着添加到知识库的信息增长而增长，对于任意语句 α 和 β
 - 如果 $KB \models \alpha$ ，那么 $KB \wedge \beta \models \alpha$
 - 例如，假设知识库包含附加断言 β ， β 宣称世界中正好有8个无底洞，这条知识可能有助于Agent推导出附加结论，但无法推翻任意已经推导出的结论 α
- 单调性意味着只要在知识库中发现了合适的前提，就可以应用推理规则——规则的结论 “与知识库中的其余内容无关”

7.5 命题逻辑定理证明-归结证明

- **归结**，一个推理规则，当它和任何一个完备的搜索算法相结合时，可以得到完备的推理算法
- 以Wumpus世界为例，讲解归结规则的一个简单版本，考虑导出下图的步骤：

- ✓ Agent从[2,1]返回[1,1]，接着走到[1,2]
- ✓ 在此地感知到臭气，但没有微风

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

$$R_{11}: \neg B_{1,2}$$

$$R_{12}: B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$$

$$R_{13}: \neg P_{2,2}$$

$$R_{14}: \neg P_{1,3}$$

$$R_{15}: P_{1,1} \vee P_{2,2} \vee P_{3,1}$$

$$R_{16}: P_{1,1} \vee P_{3,1}$$

$$R_{17}: P_{3,1}$$

7.5 命题逻辑定理证明-归结证明

■ 单元归结

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k}$$

- 其中每个 l 都是一个文字，而且 l_i 和 m 是互补文字（即，一个文字是另一个文字的否定式）。那么，单元归结规则选取一个子句——文字的析取式——和一个文字，生成一个新的子句。注意单个文字可以被视为只有一个文字的析取式，也被称为单元子句

7.5 命题逻辑定理证明-归结证明

■ 全归结

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

- 其中， ℓ_i 和 m_j 是互补文字。这说明归结选取两个子句并生成一个新的子句，该新子句包含除了两个互补文字以外的原始子句中的所有文字。示例如下：

$$\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$$

7.5 命题逻辑定理证明-归结证明

■ 合取范式

- 归结只应用于析取式（子句）。
- 对于所有命题逻辑：每个语句逻辑上都等价于某子句的合取式
- 以子句的合取式表达的语句被称为合取范式或者CNF

CNF 语句 $\rightarrow$ 子句₁ $\wedge \cdots \wedge$ 子句_n
 子句 $\rightarrow$ 文字₁ $\vee \cdots \vee$ 文字_m
 事实 $\rightarrow$ 符号
 文字 $\rightarrow$ 符号 $| \neg$ 符号
 符号 $\rightarrow P | Q | R | \cdots$
 霍恩子句范式 $\rightarrow$ 确定子句范式 $|$ 目标子句范式
 确定子句范式 $\rightarrow Fact | (符号_1 \wedge \cdots \wedge 符号_l) \Rightarrow 符号$
 目标子句范式 $\rightarrow (符号_1 \wedge \cdots \wedge 符号_l) \Rightarrow False$

7.5 命题逻辑定理证明-归结证明

- 通过将语句 $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ 转换为CNF来阐明这一过程：

(1) 消去 $\Leftrightarrow$ ，将 $\alpha \Leftrightarrow \beta$ 替换为 $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$ ：

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

(2) 消去 $\Rightarrow$ ，将 $\alpha \Rightarrow \beta$ 替换为 $\neg \alpha \vee \beta$ ：

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg (P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

(3) CNF 要求 $\neg$ 只能在文字前出现，因此我们反复应用图 7-11 的如下等价关系“将 $\neg$ 内移”：

$$\neg(\neg \alpha) \equiv \alpha \quad (\text{双重否定律})$$

$$\neg(\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta) \quad (\text{德摩根律})$$

$$\neg(\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta) \quad (\text{德摩根律})$$

本例中，我们只需运用最后一条规则一次：

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

(4) 现在我们得到了一个 $\wedge$ 和 $\vee$ 嵌套、运算符直接作用于文字的语句。运用图 7-11 的分配律，尽可能地对 $\wedge$ 分配 $\vee$ ：

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

7.5 命题逻辑定理证明-归结证明

■ 基于归结的推理过程使用的是反证法证明原理

□ 例：若要证明 $KB \models \alpha$ ，则证 $KB \wedge \neg \alpha$ 是不可满足的

function PL-RESOLUTION(KB, a) **returns** true或false

inputs: KB ，知识库，一个命题逻辑中的语句

a ，查询，一个命题逻辑中的语句

$clauses \leftarrow KB \wedge \neg a$ 的CNF表示的子句集合

$new \leftarrow \{\}$

while true **do**

for each 子句对 C_i, C_j **in** $clauses$ **do**

$resolvents \leftarrow \text{PL-RESOLVE}(C_i, C_j)$

if $resolvents$ 包含空子句 **then return** true

$new \leftarrow new \cup resolvents$

if $new \subseteq clauses$ **then return** false

$clauses \leftarrow clauses \cup new$

图 7-13 简单的命题逻辑归结算法。PL-RESOLVE 返回对其两个输入进行归结得到的所有可能子句集合

7.5 命题逻辑定理证明-归结证明

- 对 wumpus 世界的一个简单推断部分运用 PL-Resolution 来证明查询 $\neg P_{1,2}$

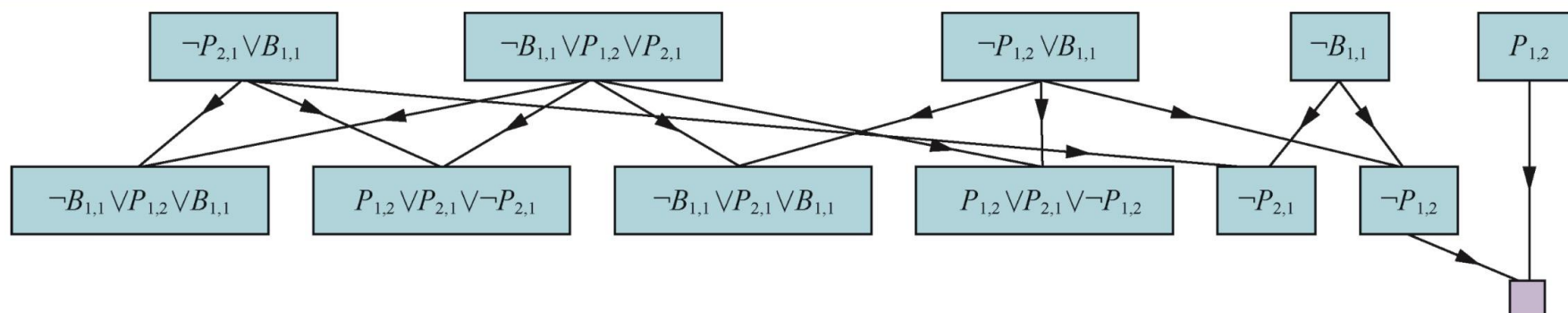


图 7-14 对 wumpus 世界的一个简单推断部分运用 PL-RESOLUTION 来证明查询 $\neg P_{1,2}$ 。顶行最左侧的 4 个子句的每一个与其他 3 个都互相成对，运用归结规则产生底行的子句。顶行的第 3 个和第 4 个子句结合生成 $\neg P_{1,2}$ ，它继而与 $P_{1,2}$ 归结，生成空子句，表明查询被证明

- 第二行给出了对第一行进行归结得到的所有子句
- 当 $P_{1,2}$ 与 $\neg P_{1,2}$ 进行归结时得到空子句，用小方框表示
- 注意：可以删除同时出现两个互补文字的任何子句

7.5 命题逻辑定理证明-归结证明

■ 归结的完备性

- 子句集 S 的归结闭包 $RC(S)$ ，通过对 S 中的子句或其派生子句反复应用归结规则而生成的所有子句的集合
- PL-RESOLUTION计算的归结闭包是变量clauses的最终值
 - ▶ $RC(S)$ 一定是有限的，因为 S 中出现的符号 $P_1, \dots, P_k$ 只能构成有限多个不同的子句，所有PL-RESOLUTION可终止

7.5 命题逻辑定理证明-归结证明

■ 归结的完备性

- 命题逻辑中归结的完备性定理被称为基本归结定理
 - ▶ 如果子句集是不可满足的, 那么这些子句的归结闭包包含空子句
- 证明其逆否命题, 如果闭包 $RC(S)$ 不包含空子句, 那么 S 是可满足的。
可以用 $P_1, \dots, P_k$ 的适当真值构造 S 的模型。

对于 i 从 1 到 k ,

- 如果 $RC(S)$ 中的子句含有文字 $\neg P_i$ 且所有其他文字在对 $P_1, \dots, P_{i-1}$ 选定的赋值下为假, 则对 P_i 赋值为 false;
- 否则, 对 P_i 赋值为 true。

7.5 命题逻辑定理证明-Horn子句和限定子句

- **限定子句**是指恰好只含一个正文字的析取式
 - 例如, 子句 $(\neg L_{1,1} \vee \neg Breeze \vee B_{1,1})$ 是限定子句, 而 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1})$ 不是
- 更一般的形式有Horn子句, 是指至多只有一个正文字的析取式
 - 所有限定子句都是Horn子句
 - 没有正文字的析取式也是Horn子句: 称为目标子句
 - Horn子句在归结下是封闭的: 如果对两个Horn子句进行归结, 结果依然是Horn子句

7.5 命题逻辑定理证明-Horn子句和限定子句

■ 只包含限定子句的知识库

- 每个限定子句都可以写成蕴含式，它的前提为正文字的合取式，结论为单个正文字
- 使用Horn子句的推理可以使用前向链接和反向链接算法。这种类型的推理构成了逻辑程序设计的基础
- 用Horn子句判定蕴涵需要的时间与知识库大小呈线性关系

7.5 命题逻辑定理证明-前向和反向链接

- 前向链接算法PL-FC-ENTAILS?(KB, q)
 - 判定单个命题词 q ——查询——是否被限定子句的知识库所蕴涵
 - 它从知识库中的已知事实（正文字）出发
 - 如果蕴含式的所有前提已知，那么就把它结论添加到已知事实集
 - 这个过程持续进行，直到查询 q 被添加或者无法进行更进一步的推理

7.5 命题逻辑定理证明-前向和反向链接

■ 前向链接算法PL-FC-ENTAILS?(KB, q)

function PL-FC-ENTAILS?(KB, q) **returns** true或false

inputs: KB , 知识库, 一个命题确定子句集

q , 查询, 一个命题符号

$count \leftarrow$ 一个表, 其中 $count[c]$ 为子句 c 的初始符号数量

$inferred \leftarrow$ 一个表, 其中 $inferred[s]$ 初始对所有符号为 false

$queue \leftarrow$ 一个符号队列, 初始符号为 KB 中已知为 true 的符号

while $queue$ 不空 **do**

$p \leftarrow \text{POP}(queue)$

if $p = q$ **then return** true

if $inferred[p] = \text{false}$ **then**

$inferred[p] \leftarrow \text{true}$

for each p 在 $c.PREMISE$ 中的知识库中的子句 c **do**

$count[c]$ 减 1

if $count[c] = 0$ **then** 将 $c.CONCLUSION$ 添加到 $queue$

return false

运行时间是线性的

图 7-15 命题逻辑的前向链接算法。 $queue$ 记录了已知为真但还没“处理过”的符号。表 $count$ 记录每个蕴涵式尚未证明的前提数量。一旦 $queue$ 中的新符号 p 被处理, $count$ 就会为每个前提中出现 p 的蕴涵式减 1 (使用合适的索引方法, 很容易在常数时间内完成)。如果 $count$ 为 0, 则蕴涵式的所有前提都已知, 因此其结论可以添加到 $queue$ 。最后, 我们还需要记录哪个符号已经被处理过, 如此在已推断符号集 $inferred$ 中的符号就无需被再次加入 $queue$ 。这避免了重复的工作, 也避免了由类似 $P \Rightarrow Q$ 和 $Q \Rightarrow P$ 这样的蕴涵式引起的循环

7.5 命题逻辑定理证明-前向和反向链接

■ 前向链接算法PL-FC-ENTAILS?(KB, q)

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

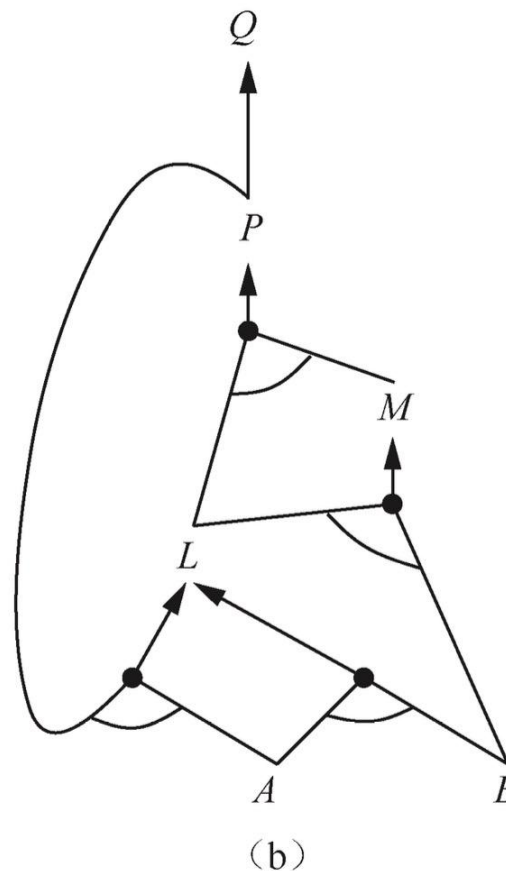
$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

A

B

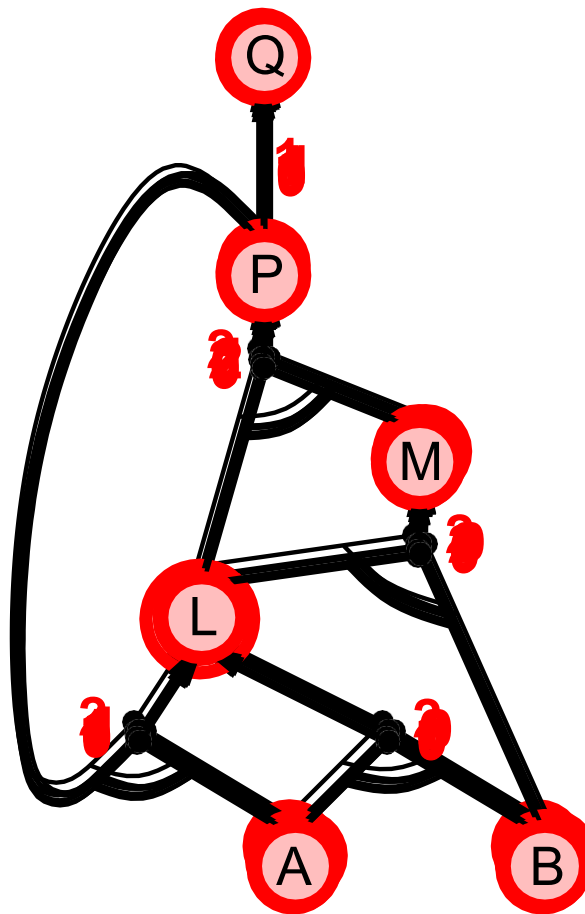
(a)



(a) 一个霍恩子句集。(b) 相应的与或图

7.5 命题逻辑定理证明-前向和反向链接

■ 前向链接示例



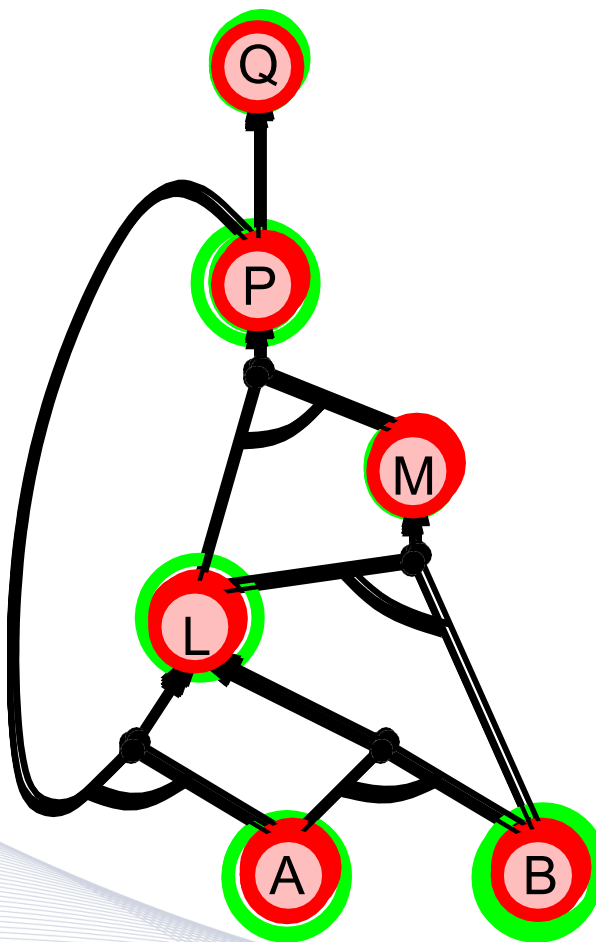
7.5 命题逻辑定理证明-前向和反向链接

■ 反向链接算法

- 如果查询 q 已知为真，则不需要做任何操作
- 否则，在知识库中找寻结论为 q 的蕴含式。
- 如果这些蕴含式的所有前提都可以（用反向链接）证明为真，则 q 为真

7.5 命题逻辑定理证明-前向和反向链接

■ 反向链接示例



- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.6 高效命题逻辑模型检验-引入

- 描述可满足性检验 (SAT) 的方法
- 许多计算科学中的组合问题都可以被归为检验命题语句的可满足性
- 两种高效的、基于模型检验的一般命题推断的算法
 - 回溯搜索
 - 局部 (爬山) 搜索算法

7.6 高效命题逻辑模型检验-完备的回溯算法

- 戴维斯-普特南算法(Davis-Putnam), 以Martin Davis和Hilary Putnam(1960)的开创性论文命名。实际上, 该算法是Davis, Logemann和Loveland(1962)描述的版本, 因此按所有四个作者的首字母缩写将其命名为DPLL
- 本质上是可能模型的递归深度优先枚举算法, 相对于TT-ENTAILS的简单方法, 改进如下:
 - **及早终止**: 可以用部分完成的模型来判断语句是否一定为真或为假, 避免了搜索空间的全部子树

如果A为真, 那么语句 $(A \vee B) \wedge (A \vee C)$ 为真, 不论B和C取何值

7.6 高效命题逻辑模型检验-完备的回溯算法

- 本质上是可能模型的递归深度优先枚举算法，相对于TT-ENTAILS的简单方法，改进如下：

- **纯符号启发式**：纯符号是指在所有子句中以相同“符号位”出现的符号

例如在这三个子句 $(A \vee \neg B)$, $(\neg B \vee \neg C)$ 和 $(C \vee A)$ 中，A、B为纯符号，C是非纯的

- **单元子句启发式**：只有一个文字的子句。在DPLL的背景下，它还表示这样的子句，即除某个文字以外的所有其他文字都被模型赋值为false

例如，如果模型包括 $B=\text{true}$ ，那么 $(\neg B \vee \neg C)$ 则成为 $\neg C$ ，即单元子句。

7.6 高效命题逻辑模型检验-完备的回溯算法

■ 用于检验命题逻辑语句可满足性的DPLL算法

function DPLL-SATISFIABLE?(*s*) **returns** true或false

inputs: *s*, 一个命题逻辑中的语句

clauses $\leftarrow$ *s*的CNF表示的子句集合

symbols $\leftarrow$ *s*中的命题符号列表

return DPLL(*clauses*, *symbols*, {})

function DPLL(*clauses*, *symbols*, *model*) **returns** true或false

if 在*model*中每个子句都为true **then return** true

if 在*model*中*clauses*中的某个子句为false **then return** false

P, *value* $\leftarrow$ FIND-PURE-SYMBOL(*symbols*, *clauses*, *model*)

if *P*不空 **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P, *value* $\leftarrow$ FIND-UNIT-CLAUSE(*clauses*, *model*)

if *P*不空 **then return** DPLL(*clauses*, *symbols* – *P*, *model* $\cup$ {*P*=*value*})

P $\leftarrow$ FIRST(*symbols*); *rest* $\leftarrow$ REST(*symbols*)

return DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=true}) **or**

DPLL(*clauses*, *rest*, *model* $\cup$ {*P*=false})

7.6 有效命题逻辑模型检验-局部搜索算法

- 目标是找出满足每个子句的变量赋值，评价函数可以选择未满足子句的数量。算法涉及完全赋值空间，该空间包括很多局部极小值点
 - ▶ 试图找出贪婪性和随机性之间的平衡点

- WalkSAT是最简洁有效的算法
 - ▶ 每次迭代中选择一个未得到满足的子句
 - ▶ 从该子句中选择要给命题符号进行翻转操作
 - “最小冲突”，最小化新状态下未得到满足语句数量
 - “随机游走”来随机挑选符号

7.6 高效命题逻辑模型检验-局部搜索算法

- 局部搜索算法举例：通过随机翻转变量的值来检验可满足性的WalkSAT算法。

function WALKSAT(*clauses*, *p*, *max_ ips*) **returns** 满足的模型或*failure*

inputs: *clauses*, 命题逻辑的子句集合

p, 选择“随机游走”动作的概率, 通常为0.5左右

max_ ips, 放弃搜索前允许的值翻转次数

model $\leftarrow$ 对*clauses*中的符号随机赋值true/false

for each *i* = 1 **to** *max_ ips* **do**

if *model* 满足*clauses* **then return** *model*

clause $\leftarrow$ 从*clauses*中随机选择的一个在*model*中为false的子句

if RANDOM(0, 1) $\leq p$ **then**

在*model*中翻转一个从*clause*中随机选择的符号的值

else 翻转*clause*中的符号以最大化已满足子句的数量

return *failure*

7.6 有效命题逻辑模型检验-随机SAT问题

- n皇后问题：解密集分布在赋值空间上，任意初始赋值都可以保证在其附近存在某个解，它是**低约束的**
 - 用回溯算法相当棘手
 - 用局部搜索算法，比如最小冲突法，求解非常容易
- 考虑合取范式的可满足性问题时，低约束的问题是具有相对较少的子句来约束变量的问题。

$$(\neg DV \neg BVC) \wedge (BV \neg AV \neg C) \wedge (\neg CV \neg BVE) \wedge (EV \neg DVB) \\ \wedge (BVEV \neg C)$$

- 32个可能的赋值中有16个是此语句的模型

7.6 高效命题逻辑模型检验-随机SAT问题

- $CNF_K(m, n)$ 表示有 m 个子句， n 个符号的 k -CNF语句，均匀独立地选择子句，每个子句中的 k 个不同文字随机选择正文字或负文字（一个符号不能在一个子句中出现两次，一个子句也不能在语句中出现两次）

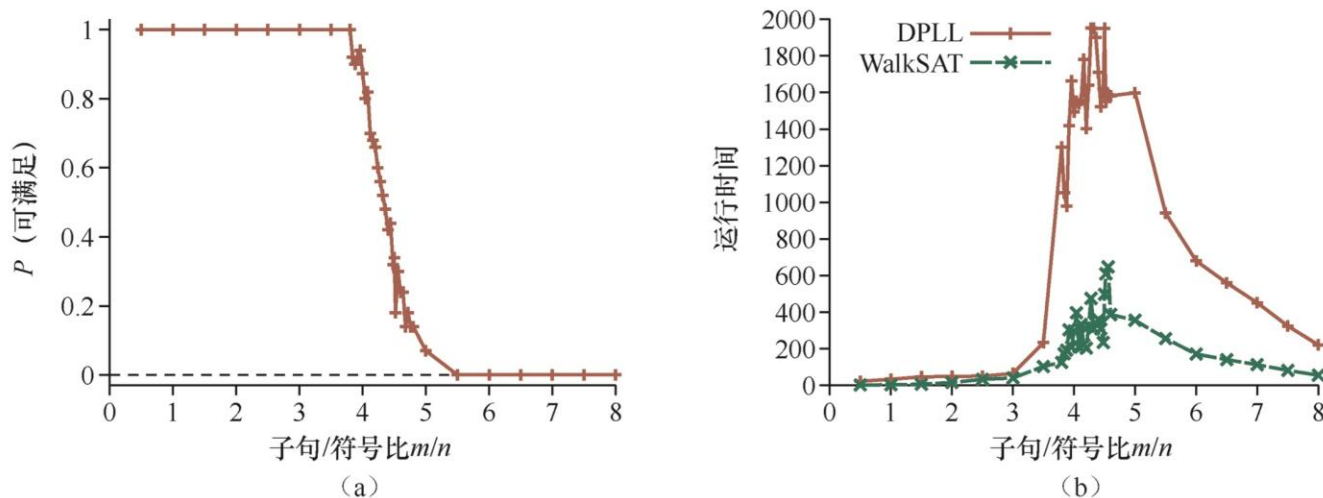


图 7-19 (a) 有 n 个符号的随机 3-CNF 语句的可满足概率图，概率是子句/符号比 m/n 的函数。(b) DPLL 和 WALKSAT 在随机 3-CNF 语句上的（多次运行后测量的）运行时间中位数图。最为困难的问题的子句/符号比约为 4.3

- 基于知识的Agent
- Wumpus世界
- 逻辑
- 命题逻辑：一种简单的逻辑
- 命题逻辑定理证明
- 高效的命题逻辑模型检验
- 基于命题逻辑的Agent

7.6 基于命题逻辑的Agent

■ Wumpus世界问题：我在哪儿、方格是否安全？

- Agent知道开始方格不包含无底洞和Wumpus
- 方格有微风当且仅当邻居方格中有无底洞
- 方格有臭气当且仅当邻居方格中有怪兽

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

$$S_{1,1} \Leftrightarrow (W_{1,2} \vee W_{2,1})$$

- 只有一只Wumpus

假定至少存在一只Wumpus: $W_{1,1} \vee W_{1,2} \vee \dots \vee W_{4,3} \vee W_{4,4}$

至多存在一只Wumpus:

$$\neg W_{1,1} \vee \neg W_{1,2}$$

$$\neg W_{1,1} \vee \neg W_{1,3}$$

...

$$\neg W_{4,3} \vee \neg W_{4,4}$$

7.6 基于命题逻辑的Agent

■ Wumpus世界问题：我在哪儿、方格是否安全？

□ 非时序变量：永远不变的符号，不需要时间上标

□ $L_{1,1}^0$ ：Agent在时间0位于[1,1]

□ 插入断言

$$L_{x,y}^t \Rightarrow (Breeze^t \Leftrightarrow B_{x,y})$$

$$L_{x,y}^t \Rightarrow (Stench^t \Leftrightarrow S_{x,y})$$

□ 对每个流F，有公理以流F在t的值和时刻t可能的行动来定义 F^{t+1} 的真值

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$$

7.6 基于命题逻辑的Agent

■ 混合Agent

- 将条件-行动规划与前述章节的问题求解算法结合在一起，得到混合Agent
- Agent算法维护和更新知识库和当前规划

7 本章小结

- 讨论了基于知识的Agent，指明了如何定义一种逻辑，使得Agent可以使用这种逻辑对世界进行推理
- Agent的知识以知识表示语言的语句形式存储在Agent的知识库中
- 基于知识的Agent由知识库和推理机制组成
 - ▶ 把描述世界的语句存储到知识库中，使用推理机制推导出新的语句，根据这些语句来决策采取什么行动

7 本章小结

- 一种表示语言是通过语法和语义来定义的，其中语法指明语句的结构，而语义定义了每个语句在每个可能世界或模型中的真值
- 语句之间的蕴涵关系对于理解推理至关重要。如果 β 语句在所有 α 语句为真的世界中都为真，则 α 蕴涵 β 。等价定义包括语句 $\alpha \Rightarrow \beta$ 是有效的和 $\alpha \wedge \neg \beta$ 是不可满足的
- 推理是从旧语句生成新语句的过程。可靠的推理算法只生成被蕴涵的语句；完备的算法生成所有被蕴涵的语句

7 本章小结

- 命题逻辑是由命题词和逻辑连接词组成的简单语言。它可以处理已知为真、已知为假或完全未知的命题
- 给定一个固定的命题词汇表，它的可能模型集是有限的，因此蕴涵可以通过枚举模型来进行检验。
- 推理规则是用来寻找证明的可靠推理模式。归结规则是用在表示为合取范式的知识库上的完备推理算法。
- 前向链接和反向链接都是用在以Horn子句表示的知识库上的自然的推理算法

7 本章小结

- 用于命题逻辑的高效模型检验推理算法包括回溯和局部搜索方法，通常可以快速求解大规模的问题。
 - ▶ 局部搜索算法如WALKSAT可以用于问题求解，是可靠的但是不完备
- 命题逻辑无法扩展到无限的环境，原因是它缺乏足够的表达能力来准确地处理时间、空间以及对象间关系的模式

课后作业

以下式子哪些是正确的？

- a. $False \models True$ 。
- b. $True \models False$ 。
- c. $(A \wedge B) \models (A \Leftrightarrow B)$ 。
- d. $A \Leftrightarrow B \models A \vee B$ 。
- e. $A \Leftrightarrow B \models \neg A \vee B$ 。
- f. $(A \wedge B) \Rightarrow C \models (A \Rightarrow C) \vee (B \Rightarrow C)$ 。
- g. $(C \vee (\neg A \wedge \neg B)) \equiv ((A \Rightarrow C) \wedge (B \Rightarrow C))$ 。
- h. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B)$ 。
- i. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B) \wedge (\neg D \vee E)$ 。
- j. $(A \vee B) \wedge \neg(A \Rightarrow B)$ 是可满足的。
- k. $(A \Leftrightarrow B) \wedge (\neg A \vee B)$ 是可满足的。
- l. $(A \Leftrightarrow B) \Leftrightarrow C$ 与包含固定集合命题符号 A, B, C 的 $(A \Leftrightarrow B)$ 的模型数相等。

课后作业

证明以下断言或举出反例。

- a. 若 $a \models \gamma$ 或 $\beta \models \gamma$ (或两者都有) 则 $(a \wedge \beta) \models \gamma$ 。
- b. 若 $a \models (\beta \wedge \gamma)$ 则 $a \models \beta$ 且 $a \models \gamma$ 。
- c. 若 $a \models (\beta \vee \gamma)$ 则 $a \models \beta$ 或 $a \models \gamma$ (或两者都成立)。

THANKS